Product Requirements Document (PRD)

Hospital Bed Allocation System

Version: 1.0
Date: 2026-05-17
Status: Draft
Author: Student Project Team
Target: College DBMS Course Project

---

1. Executive Summary

1.1 Product Vision

Build a desktop-based Hospital Bed Allocation System that demonstrates core Database Management System (DBMS) concepts through a functional, transactional application. The system manages hospital bed inventory, patient admissions, discharges, waitlists, and generates occupancy reports.

1.2 Target Users

- **Hospital Receptionist / Admin Staff**: Primary users who admit patients, allocate beds, process discharges, and view reports.
- **System Administrator**: Manages ward configurations, bed inventory, and staff records.

1.3 Success Criteria

- All beds can be tracked in real-time (Available, Occupied, Maintenance).
- Patient admission and discharge are atomic database transactions.
- The system demonstrates 3NF/BCNF normalization, triggers, stored procedures, views, and constraints.
- Waitlist is automatically processed when a bed becomes available.
- Swing GUI provides color-coded visual bed status.

---

2. Scope

2.1 In Scope

- Ward and bed inventory management
- Patient registration and admission workflow
- Bed allocation with ward-type preference
- Patient discharge and bed deallocation
- Waitlist management with priority scoring
- Bed maintenance scheduling
- Staff (doctor/nurse) assignment tracking
- Occupancy and turnover reports
- Audit trail for bed status changes

2.2 Out of Scope

- Billing and payment processing
- Electronic Medical Records (EMR) / clinical notes
- Pharmacy inventory management
- Multi-hospital deployment
- Web/mobile interfaces
- Role-based access control beyond basic admin/receptionist
- Real-time IoT sensor integration

---

3. Functional Requirements

3.1 Ward & Bed Management (FR-BED)

ID	Requirement	Priority
FR-BED-01	System shall store ward details: Ward ID, Ward Type (General, ICU, Emergency, Private), Floor, Capacity, Daily Charge	High
FR-BED-02	System shall store bed details: Bed ID, Ward ID, Bed Number, Equipment Status, Current Status (Available, Occupied, Maintenance)	High
FR-BED-03	System shall allow adding new wards and beds via admin panel	Medium
FR-BED-04	System shall allow marking a bed as “Maintenance” and logging the reason	Medium
FR-BED-05	System shall prevent allocating a bed that is Occupied or under Maintenance	High

3.2 Patient Management (FR-PAT)

ID	Requirement	Priority
FR-PAT-01	System shall store patient details: Patient ID, Name, Age, Blood Group, Contact, Address, Emergency Contact	High
FR-PAT-02	System shall allow registering a new patient before admission	High
FR-PAT-03	System shall maintain patient admission history (multiple admissions over time)	High

Table 2 – continued

ID	Requirement	Priority
FR-PAT-04	System shall support searching patients by ID or Name	Medium

3.3 Admission & Allocation (FR-ADM)

ID	Requirement	Priority
FR-ADM-01	System shall allow admitting a patient by selecting ward type preference	High
FR-ADM-02	System shall automatically find and allocate the first available bed matching the ward type	High
FR-ADM-03	System shall record admission timestamp, expected discharge date, assigned doctor, and bed ID	High
FR-ADM-04	System shall atomically update bed status to "Occupied" upon successful admission	High
FR-ADM-05	System shall handle concurrent allocation requests without double-booking (transaction isolation)	High
FR-ADM-06	If no bed is available, system shall add patient to waitlist with priority score	High
FR-ADM-07	Priority score = (Severity Level × 100) + Wait Time in hours	Medium

3.4 Discharge & Deallocation (FR-DIS)

ID	Requirement	Priority
FR-DIS-01	System shall allow discharging a patient from an occupied bed	High
FR-DIS-02	System shall record actual discharge timestamp and calculate stay duration	High
FR-DIS-03	System shall atomically free the bed (status → Available or Maintenance)	High

Table 4 – continued

ID	Requirement	Priority
FR-DIS-04	Upon discharge, system shall automatically check waitlist and allocate bed to highest-priority waiting patient of matching ward type	High
FR-DIS-05	System shall generate a discharge summary with bed turnover time	Medium

3.5 Waitlist Management (FR-WAI)

ID	Requirement	Priority
FR-WAI-01	System shall display waitlist sorted by priority score (descending)	High
FR-WAI-02	System shall allow manual removal of a patient from waitlist	Medium
FR-WAI-03	System shall show estimated wait time based on average turnover per ward type	Low
FR-WAI-04	When a bed is freed, system shall auto-allocate to the highest-priority waitlisted patient for that ward type	High

3.6 Staff Management (FR-STF)

ID	Requirement	Priority
FR-STF-01	System shall store staff: Staff ID, Name, Role (Doctor/Nurse), Department, Phone	Medium
FR-STF-02	System shall link a doctor to each admission record	Medium
FR-STF-03	System shall display doctor workload (number of current admissions per doctor)	Low

3.7 Reporting & Analytics (FR-RPT)

ID	Requirement	Priority
FR-RPT-01	Dashboard showing total beds, occupied count, available count, maintenance count per ward	High
FR-RPT-02	Current occupancy rate per ward type (percentage)	High
FR-RPT-03	Bed turnover report: average stay duration per ward type	Medium
FR-RPT-04	Waitlist summary: count per ward type, longest waiting patient	Medium
FR-RPT-05	Doctor workload report	Low

3.8 Audit & Logging (FR-AUD)

ID	Requirement	Priority
FR-AUD-01	Every bed status change shall be logged with timestamp, old status, new status, and triggering event	Medium
FR-AUD-02	Admission and discharge events shall be immutable records	High

---

4. Non-Functional Requirements

4.1 Performance (NFR-PERF)

- Bed allocation query shall execute within 2 seconds for up to 500 beds.
- Dashboard data refresh shall take < 3 seconds.
- Waitlist auto-allocation trigger shall execute within 1 second of discharge.

4.2 Reliability (NFR-REL)

- Database transactions for admission/discharge must be ACID-compliant.
- No data loss on concurrent allocation attempts.

4.3 Usability (NFR-UI)

- Swing GUI shall use color coding: Green (Available), Red (Occupied), Yellow (Maintenance), Gray (Other).
- All forms shall have input validation with clear error messages.
- Table views shall support sorting by clicking column headers.

4.4 Maintainability (NFR-MNT)

- SQL schema, triggers, and procedures shall be stored in separate `.sql` files.
- Java code shall follow DAO pattern with clear separation of GUI and database layers.

4.5 Portability (NFR-PORT)

- System shall run on any machine with JDK 17+ and MySQL 8.0+ / PostgreSQL 14+.
- Database connection parameters shall be externalized in a config file.

---

5. User Interface Requirements

5.1 Screens

1. **Login Screen**: Username/password (simplified for college demo).
2. **Main Dashboard**:
   - Top bar: Total stats (Total Beds / Occupied / Available / Maintenance).
   - Center: `JTable` of all beds with color-coded rows.
   - Side panel: Quick action buttons (Admit, Discharge, Waitlist, Reports).
3. **Admit Patient Dialog**:
   - Patient search/registration form.
   - Ward type preference dropdown.
   - Doctor assignment dropdown.
   - "Allocate Bed" button with confirmation.
4. **Discharge Dialog**:
   - List of currently occupied beds with patient names.
   - "Discharge" button.
   - Post-discharge: option to mark bed for maintenance.
5. **Waitlist Manager**:
   - `JTable` of waitlisted patients with priority scores.
   - "Auto-Allocate" and "Remove" buttons.
6. **Reports Screen**:
   - Tabbed interface: Occupancy, Turnover, Doctor Workload.
   - Each tab displays a `JTable` populated from database Views.
7. **Admin Panel**:
   - Forms to add/edit wards, beds, and staff.

5.2 GUI Standards

- Use `JTable` with custom `TableCellRenderer` for color coding.
- Use `JOptionPane` for confirmations and error alerts.
- Use `JComboBox` for dropdown selections (ward type, doctor, bed status).
- Use `JDatePicker` or `JFormattedTextField` for date inputs.
- Window size: 1024×768 minimum, resizable.

---

6. Data Requirements

6.1 Data Entities

- **Ward**: ward_id, ward_type, floor, capacity, daily_charge
- **Bed**: bed_id, ward_id, bed_number, equipment_status, current_status
- **Patient**: patient_id, name, age, blood_group, contact, address, emergency_contact

- **Staff**: staff_id, name, role, department, phone
- **Admission**: admission_id, patient_id, bed_id, doctor_id, admission_date, expected_discharge, actual_discharge, status
- **Waitlist**: waitlist_id, patient_id, requested_ward_type, priority_score, request_time, status
- **BedMaintenanceLog**: log_id, bed_id, start_time, end_time, reason, resolved_by

6.2 Data Volumes (Expected)

- Wards: 4–10 records
- Beds: 20–100 records
- Patients: 50–500 records
- Admissions: 100–1000 records (historical + active)
- Staff: 10–50 records
- Waitlist: 0–20 active records at any time

---

7. Constraints & Assumptions

7.1 Constraints

- Single-user or single-workstation deployment (college demo scope).
- No network authentication (local DB user).
- No encryption required for demo data.
- Swing is the only permitted GUI framework.

7.2 Assumptions

- Hospital operates 24/7; system does not need shift scheduling.
- One patient per bed at any given time.
- Bed equipment status is manually updated by admin.
- Severity level is manually entered by receptionist (1–5 scale).

---

8. Risks

Risk	Mitigation
Concurrent bed allocation race condition	Use JDBC transactions with proper isolation level (SERIALIZABLE or pessimistic locking)
Database connection failure	Implement connection retry logic and show user-friendly error dialog
Swing GUI becomes unresponsive during DB operations	Run DB queries in `SwingWorker` background threads
Data inconsistency from manual status updates	Enforce bed status changes only through triggers/ procedures

---

9. Deliverables

1. **Source Code**: Java Swing application with JDBC DAO layer.
2. **SQL Scripts**: `schema.sql`, `triggers.sql`, `procedures.sql`, `views.sql`, `seed_data.sql`.
3. **Database Design Document**: ER Diagram, Normalization steps (UNF → BCNF), FDs.
4. **Project Report**: Screenshots, SQL query samples, DBMS concepts explanation.
5. **Executable JAR**: Runnable demo application.

---

10. Glossary

Term	Definition
Ward Type	Category of hospital ward: General, ICU, Emergency, Private
Bed Status	Current state: Available, Occupied, Maintenance
Priority Score	Computed value determining waitlist order
Turnover Time	Duration between discharge and next admission for a bed
DAO	Data Access Object —Java pattern for database operations
ACID	Atomicity, Consistency, Isolation, Durability

End of PRD